



UNIVERSIDAD POLITÉCNICA INTERNACIONAL

DOCUMENTACIÓN TÉCNICA SISTEMA DE QUINIELAS MUNDIALISTAS V2

Iteración 2 - Aplicación web en Blazor con base de datos relacional

Estudiante: Kendall Mata Sánchez

Curso: Técnicas de Programación

Profesor: Luis Felipe Mora Umaña

Fecha: 26 de agosto de 2026

Índice

1. Introducción.....	3
2. Objetivos.....	3
3. Alcance de la Iteración 2.....	3
4. Decisiones de diseño y arquitectura.....	4
5. Base de datos relacional.....	5
6. Funcionalidades implementadas.....	6
7. Testing y cobertura.....	7
8. Instalación y ejecución.....	8
9. Guía de uso.....	9
10. Análisis de resultados.....	9
11. Aprendizajes.....	10
12. Conclusiones y recomendaciones.....	10

1. Introducción

El Sistema de Quinielas Mundialistas V2 corresponde al incremento de versión del proyecto de quinielas desarrollado previamente. La nueva iteración sustituye la capa de presentación basada en Windows Forms por una aplicación web construida con Blazor y reemplaza la persistencia anterior basada en archivos por una base de datos relacional SQLite administrada mediante Entity Framework Core.

La versión incorpora separación de roles, administración de partidos y usuarios, pronósticos, notificaciones, ranking, insignias, quinielas y reportes diferenciados. Además, los reportes pueden descargarse en CSV, TXT y PDF. La lógica de negocio se respalda mediante una suite automatizada de pruebas unitarias.

El propósito de este documento es describir las decisiones de diseño, arquitectura, estructura de datos, funcionalidades, procedimiento de ejecución, pruebas realizadas y principales aprendizajes obtenidos durante la Iteración 2.

2. Objetivos

2.1 Objetivo general

Desarrollar una segunda versión web del Sistema de Quinielas Mundialistas aplicando programación orientada a objetos, Clean Code, separación de responsabilidades y persistencia relacional, conservando las funcionalidades esenciales de la primera iteración e incorporando las mejoras solicitadas.

2.2 Objetivos específicos

- Migrar la capa de presentación a Blazor para disponer de una interfaz web.
- Sustituir la persistencia basada en archivos por una base de datos relacional SQLite.
- Implementar autenticación y separación entre los roles Usuario y Administrador.
- Permitir al administrador actualizar partidos, horarios y mantener usuarios.
- Implementar notificaciones de partidos pendientes de pronóstico en las próximas 24 horas.
- Distribuir los reportes según el rol y permitir su descarga en CSV, TXT y PDF.
- Mantener ranking, insignias y quinielas, integrándolos con la nueva persistencia.
- Validar la lógica de negocio mediante pruebas automatizadas y cobertura de código.

3. Alcance de la Iteración 2

Área	Incremento implementado
View	Migración de Windows Forms a Blazor Web.
Persistencia	SQLite + Entity Framework Core; los JSON de V1 no se usan en tiempo de ejecución.
Seguridad	Login, sesión, roles y contraseñas con PBKDF2 + salt.
Administrador	Actualización de partidos/horarios, marcador/anotadores y mantenimiento de usuarios.
Usuario	Pronósticos, historial, notificaciones, ranking, insignias, quinielas y reportes.

Reportes	Reportes por rol y exportación CSV, TXT y PDF.
Testing	xUnit + Coverlet sobre la capa de lógica de negocio.

4. Decisiones de diseño y arquitectura

4.1 Arquitectura

La solución organiza la aplicación en una capa de presentación Blazor, servicios de lógica de negocio y acceso a datos mediante Entity Framework Core. Esta separación reduce el acoplamiento entre la interfaz y la persistencia, facilita las pruebas y permite modificar una capa con menor impacto sobre las demás.



Figura 1. Flujo general de la arquitectura V2.

4.2 Programación orientada a objetos y buenas prácticas

- Encapsulamiento: las reglas de autenticación, partidos, pronósticos, reportes, ranking y administración se concentran en servicios específicos.
- Abstracción: AppDbContext y los servicios ocultan detalles de persistencia a los componentes Blazor.
- Responsabilidad única: se separan servicios como AuthService, PartidoService, PronosticoService, RankingService, NotificacionService y ReporteExportService.
- Nombres significativos: clases y métodos expresan la operación de negocio que realizan.
- Métodos enfocados: la lógica se divide en operaciones concretas para facilitar lectura, prueba y mantenimiento.
- Manejo de validaciones: se controlan escenarios como goles negativos, partidos iniciados, usuarios inactivos y restricciones por rol.

4.3 Patrones y principios

La solución utiliza una estructura compatible con MVC/separación por capas: los componentes Razor cumplen la función de View, los Services concentran el comportamiento equivalente al controlador/lógica de negocio y los modelos junto con Entity Framework Core representan el dominio y la persistencia.

En la migración de datos se conserva el enfoque Repository para desacoplar la fuente de datos, reemplazando la persistencia JSON por Entity Framework Core. La inyección de dependencias de Blazor/.NET permite que los servicios reciban sus colaboradores sin construirlos directamente, favoreciendo mantenibilidad y pruebas.

5. Base de datos relacional

La Iteración 2 utiliza SQLite mediante Microsoft.EntityFrameworkCore.Sqlite. La base principal se almacena como Data/quinielas_v2.db. Los archivos JSON de la primera iteración no forman parte de la persistencia utilizada en tiempo de ejecución por la V2.

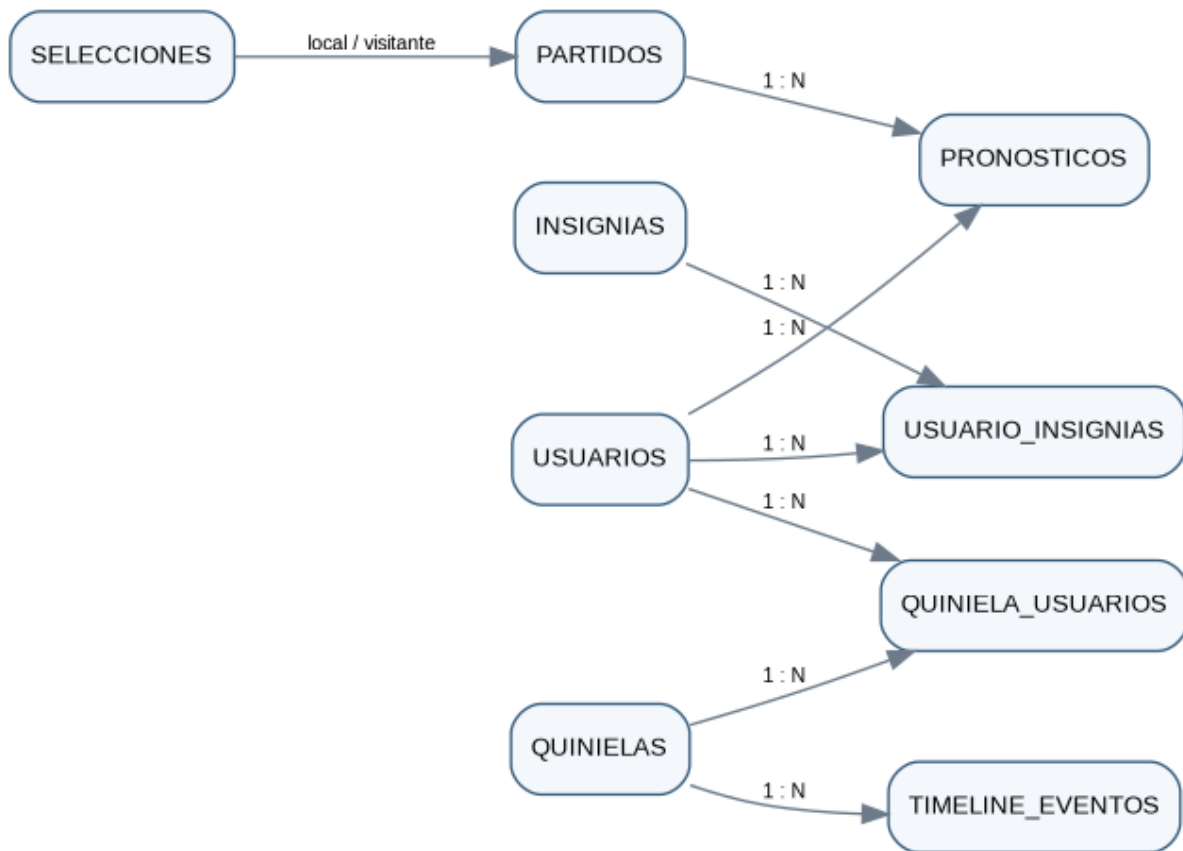


Figura 2. Diagrama relacional simplificado de la base de datos V2.

5.1 Entidades principales

Entidad	Responsabilidad
Usuarios	Credenciales, rol, estado, puntos y datos del participante.
Selecciones	Equipos participantes del torneo.
Partidos	Fecha/hora, selecciones, estado, marcador y anotadores.
Pronosticos	Marcador pronosticado, usuario, partido y puntos obtenidos.
Quinielas	Agrupaciones públicas o privadas de participantes.
QuinielaUsuarios	Relación muchos-a-muchos entre quinielas y usuarios.
Insignias / UsuarioInsignias	Catálogo y asignación de reconocimientos.
TimelineEventos	Eventos asociados a la actividad de las quinielas.
ConfiguracionSistema	Fecha simulada empleada para validar estados y escenarios.

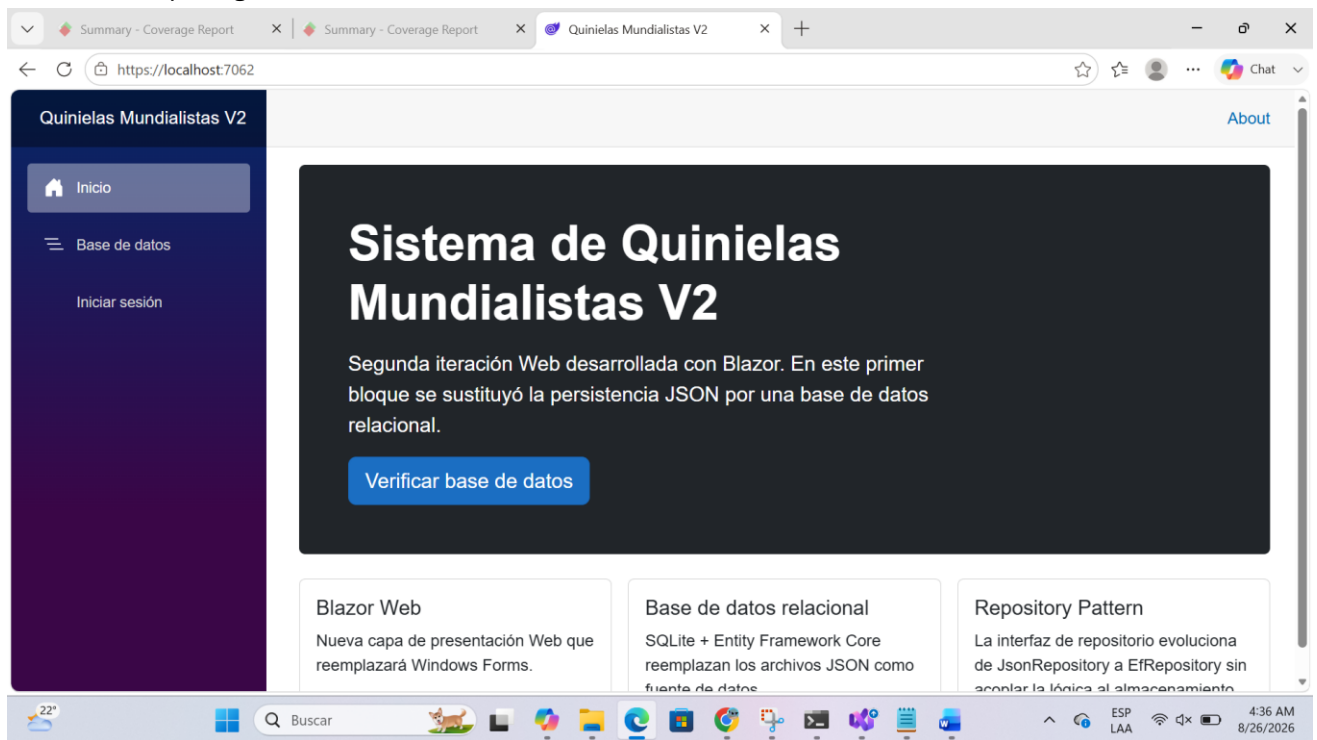
5.2 Datos migrados

La documentación interna del proyecto registra la migración inicial de 41 usuarios, 41 partidos, 210 pronósticos, 6 quinielas y 25 eventos del timeline, además de relaciones de participantes, insignias y fecha simulada.

6. Funcionalidades implementadas

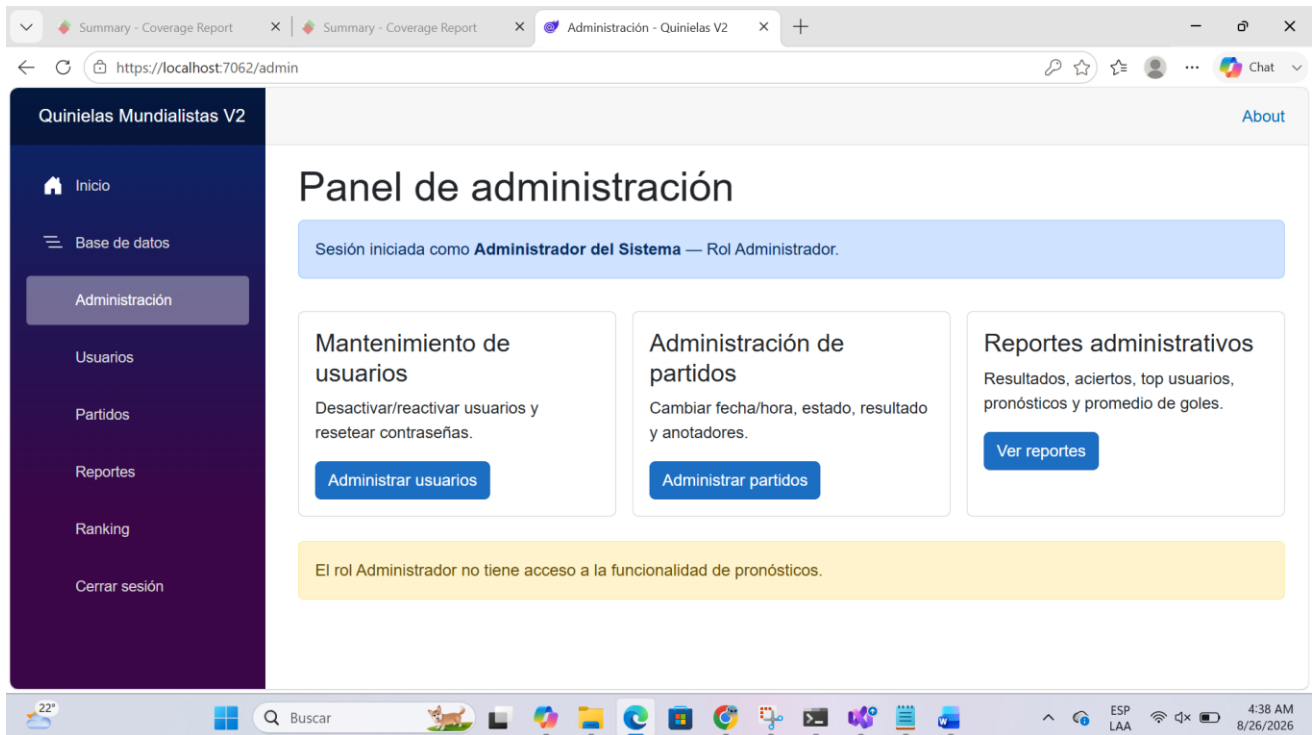
6.1 Autenticación y roles

- Inicio y cierre de sesión contra la base de datos.
- Separación entre Usuario y Administrador.
- Dashboard diferenciado según el rol.
- El administrador no dispone de opciones para registrar pronósticos.
- Contraseñas protegidas mediante PBKDF2 con salt.



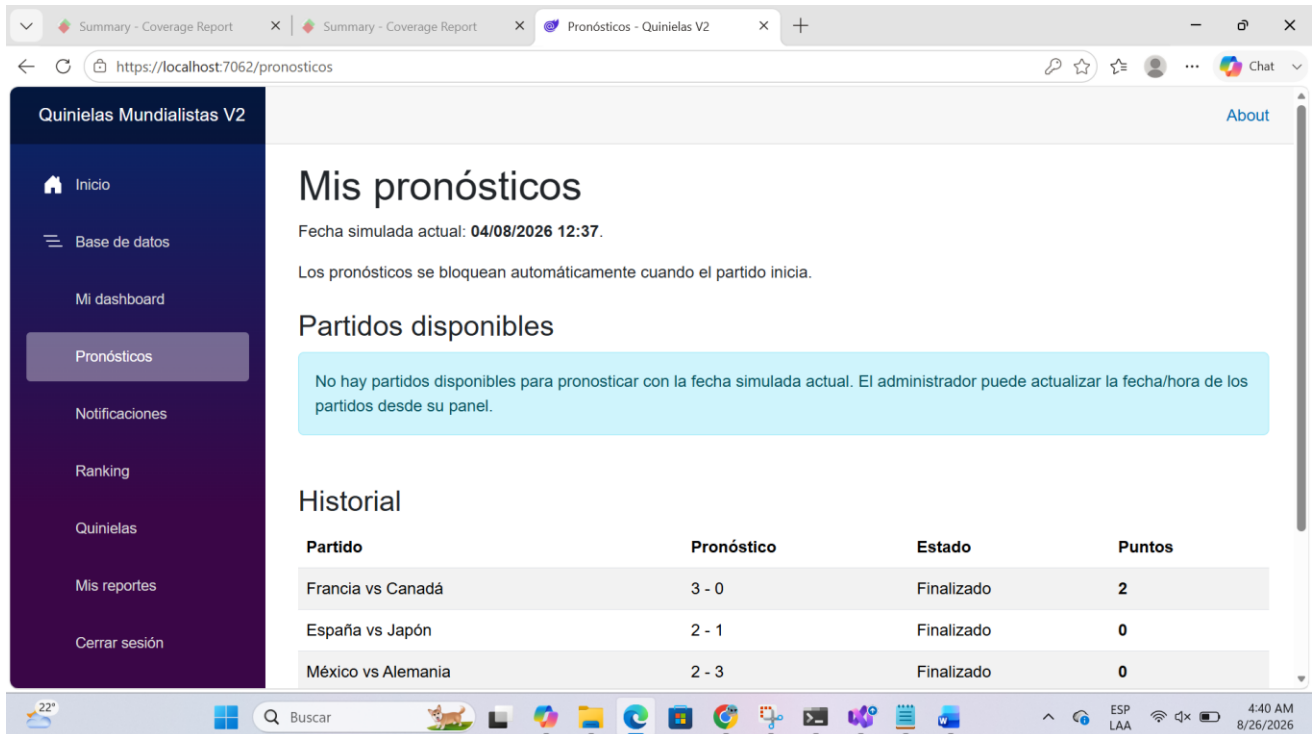
6.2 Administración

- Cambio de fecha y hora de los partidos.
- Cambio de estado: Próximo, En curso o Finalizado.
- Registro de marcador final y anotadores.
- Recalculo de puntos al finalizar un partido.
- Desactivación/reactivación de usuarios y reset de contraseña.
- Modificación de la fecha simulada para probar el comportamiento temporal.



6.3 Pronósticos y notificaciones

- Un usuario puede registrar un único pronóstico por partido.
- No se aceptan pronósticos una vez iniciado, en curso o finalizado el encuentro.
- Marcador exacto: 5 puntos; ganador o empate correcto: 2 puntos; incorrecto: 0 puntos.
- Al iniciar la sesión se consultan partidos de las próximas 24 horas sin pronóstico del usuario.
- Se excluyen de las notificaciones los partidos ya iniciados, en curso, finalizados o ya pronosticados.



6.4 Ranking, insignias y quinielas

El ranking global ordena usuarios activos por puntos, muestra sus insignias y destaca al usuario autenticado. Las insignias se muestran tanto en el dashboard como en el ranking. Las quinielas públicas son visibles para los usuarios y las privadas se restringen a sus integrantes, mostrando participantes y posiciones.

Pos.	Usuario	Nombre	País	Puntos	Insignias
1	usuario21	Aficionado 21	Costa Rica	16	Lider global, Lider privado - Quiniela Privada 3, Lider privado - Quiniela Privada 4, Lider privado - Quiniela Privada 5
2	usuario22	Aficionado 22	Argentina	13	Sin insignias
3	usuario2	Aficionado 2	Argentina	11	Lider privado - Quiniela Privada 1
4	usuario29	Aficionado 29	Canadá	11	Sin insignias
5	usuario26	Aficionado 26	Francia	10	Sin insignias

6.5 Reportes por rol

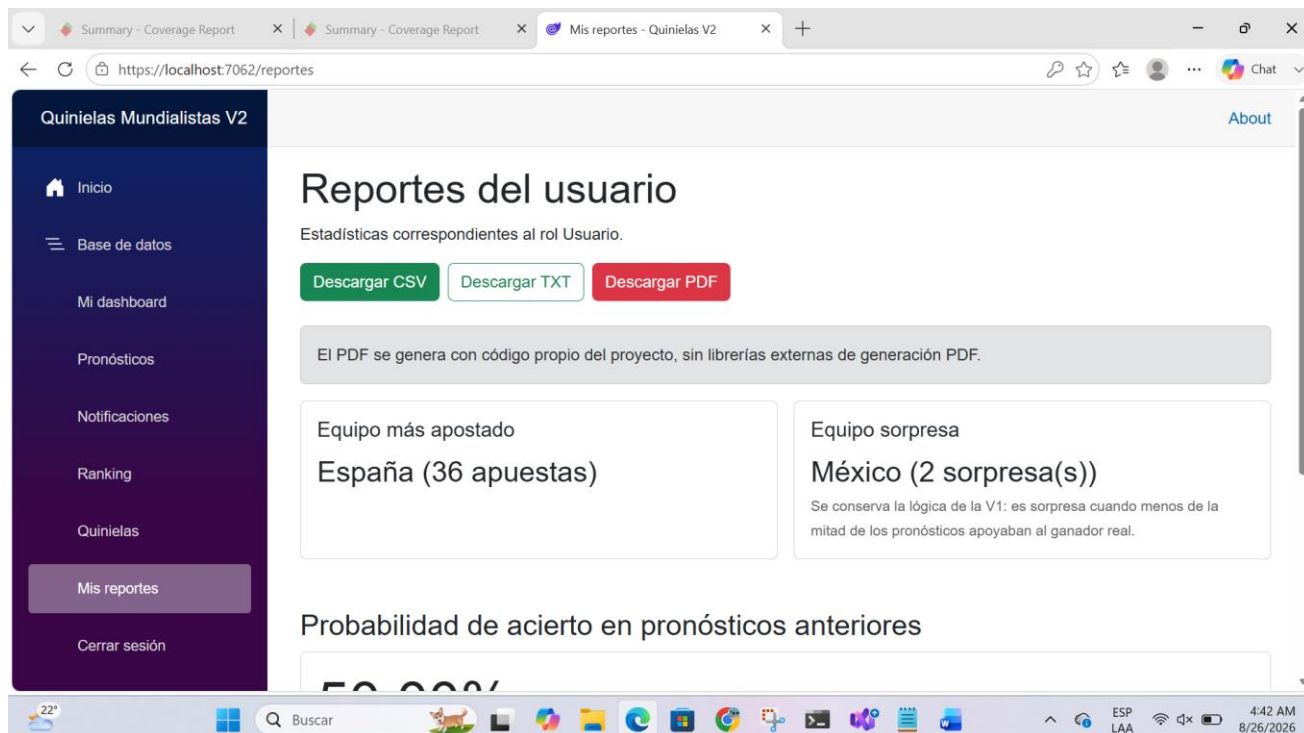
Administrador	Usuario
Resultado más repetido	Equipo más apostado
Partido con más aciertos	Equipo sorpresa con estadística
Usuarios con más aciertos: Top 1, 3 y 5	Probabilidad histórica de acierto
Partidos con más pronósticos	Cantidad de marcadores exactos
Promedio de goles	-
Partido sin aciertos	-

La probabilidad histórica de acierto se calcula como: $\text{aciertos} / \text{pronósticos de partidos finalizados} \times 100$. Se considera acierto un pronóstico que obtuvo 2 o 5 puntos.

6.6 Descarga de reportes

Los reportes pueden descargarse en CSV y TXT. Como funcionalidad adicional se implementó exportación PDF mediante código propio en `ReporteExportService`, sin depender de un paquete externo de generación PDF. El servicio construye el contenido en memoria y la interfaz Blazor inicia la descarga desde el navegador.

- CSV: separado por punto y coma para facilitar apertura con configuraciones regionales en español.
- TXT: resumen legible del reporte.
- PDF: generador simple desarrollado dentro del proyecto.



7. Testing y cobertura

La capa de lógica de negocio se prueba con xUnit y la cobertura se obtiene mediante Coverlet. Para la medición enfocada en la lógica unitariamente testeable se excluyeron los componentes Razor y Program.cs, ya que la especificación solicita cobertura de controlador o lógica de negocios.

Métrica	Resultado
Pruebas ejecutadas	72
Pruebas correctas	72
Pruebas con error	0
Cobertura de líneas	97.7%
Cobertura de ramas	87.1%

La medición final de la capa Services alcanzó 97.7% de cobertura de líneas y 87.1% de cobertura de ramas. Estos resultados superan el mínimo de 60% solicitado y también el umbral de 80% indicado para la bonificación de testing.

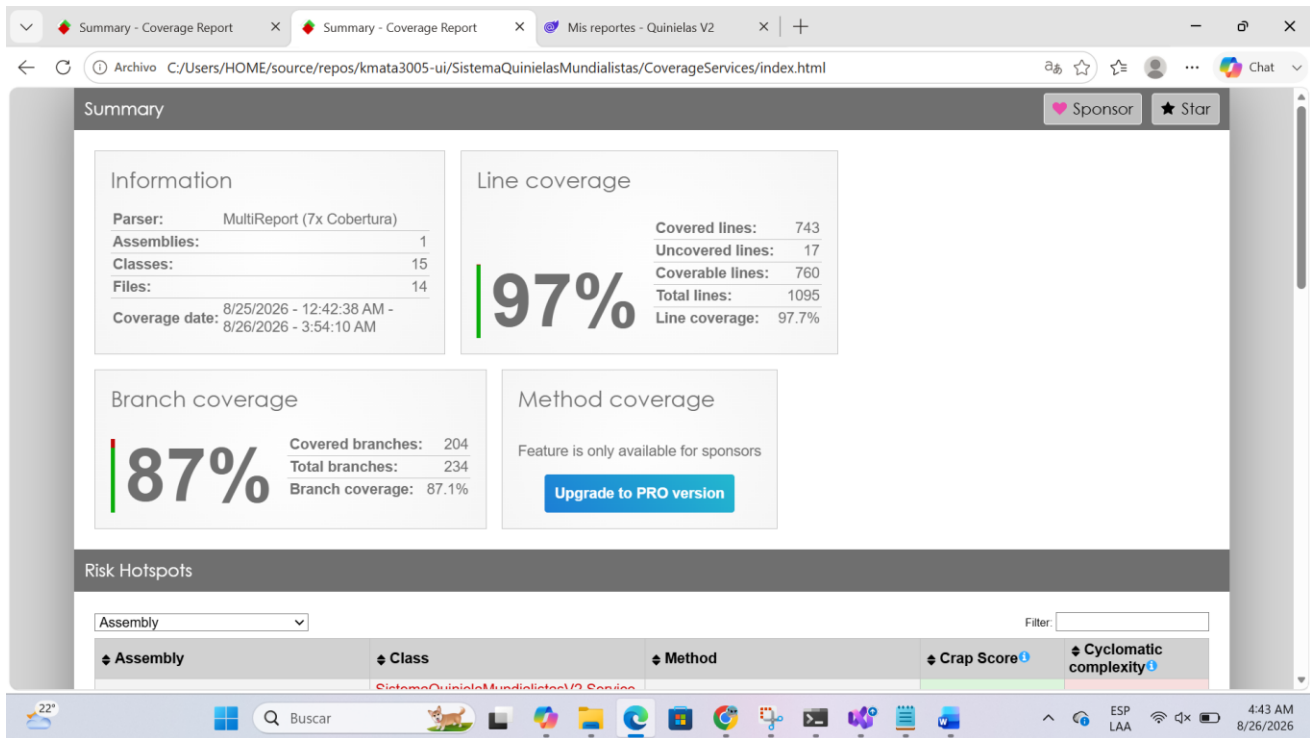


Figura 3. Resumen de evidencia final: 72 pruebas correctas, 97.7% de cobertura de líneas y 87.1% de cobertura de ramas.

8. Instalación y ejecución

8.1 Requisitos

- .NET 10 SDK.
- Visual Studio 2022/versión compatible con .NET 10 o CLI de .NET.
- Navegador web moderno.
- Repositorio del proyecto descargado o clonado.

8.2 Ejecución desde Visual Studio

1. Abrir la solución del proyecto.
2. Establecer SistemaQuinielaMundialistasV2 como proyecto de inicio.
3. Restaurar paquetes NuGet si Visual Studio lo solicita.
4. Compilar la solución.
5. Ejecutar mediante HTTPS y abrir la dirección mostrada por Visual Studio.

8.3 Ejecución desde terminal

```
dotnet restore
```

```
dotnet run --project .\SistemaQuinielaMundialistasV2.csproj
```

```
dotnet test .\SistemaQuinielaMundialistasV2.Tests\SistemaQuinielaMundialistasV2.Tests.csproj
```

8.4 Credenciales de demostración

Rol	Usuario	Contraseña
Administrador	admin	Admin2026!
Usuario	usuario1	Mundial2026

9. Guía de uso

9.1 Usuario

6. Ingresar con una cuenta de rol Usuario.
7. Revisar el dashboard, puntos, insignias y notificaciones.
8. Entrar a Pronósticos y registrar el marcador de encuentros disponibles.
9. Consultar Ranking y Quinielas.
10. Abrir Reportes de usuario para consultar equipo más apostado, equipo sorpresa y probabilidad de acierto.
11. Descargar el reporte en el formato disponible.

9.2 Administrador

12. Ingresar con la cuenta Administrador.
13. Administrar usuarios: reset de contraseña, desactivar o reactivar.
14. Administrar partidos: fecha, hora, estado, marcador y anotadores.
15. Consultar los reportes administrativos.
16. Descargar reportes en CSV, TXT o PDF.
17. Utilizar la fecha simulada cuando sea necesario validar escenarios temporales.

10. Análisis de resultados

La migración a Blazor y Entity Framework Core permitió transformar la solución de escritorio en una aplicación web con persistencia relacional, manteniendo la lógica central de la quiniela. La separación de responsabilidades mediante servicios facilitó la incorporación de nuevas reglas sin concentrar toda la lógica en la interfaz.

Las pruebas automatizadas permitieron detectar defectos concretos en el recálculo de puntajes y en escenarios de reportes. Después de las correcciones, la suite alcanzó 72 de 72 pruebas correctas. La medición final enfocada en la capa de lógica de negocio (Services) alcanzó 97.7% de cobertura de líneas y 87.1% de cobertura de ramas.

La distribución de reportes por rol y la incorporación de exportaciones fortalecen el uso práctico del sistema. La fecha simulada también resultó útil para reproducir escenarios de partidos próximos, en curso y finalizados sin depender de la fecha real.

11. Aprendizajes

- La migración de una aplicación de escritorio a web exige separar con mayor claridad la interfaz de la lógica de negocio.

- Entity Framework Core simplifica el acceso relacional, pero requiere diseñar correctamente entidades, relaciones y ciclo de vida del contexto.
- Las pruebas unitarias no solo sirven para medir cobertura: ayudaron a descubrir comportamientos incorrectos que no eran evidentes visualmente.
- Una cobertura alta debe interpretarse según el alcance medido; por ello se documenta que el porcentaje reportado corresponde a la lógica de negocio y no a los componentes de interfaz excluidos.
- El historial de Git y la planificación en Jira/Trello permiten demostrar el proceso incremental y no únicamente el resultado final.
- Las advertencias de dependencias deben tratarse como deuda técnica aun cuando el proyecto compile y las pruebas sean exitosas.

12. Conclusiones y recomendaciones

La Iteración 2 cumple con el cambio a entorno web, el uso de una base de datos relacional, la incorporación del rol administrador, notificaciones, reportes diferenciados, exportación de resultados y pruebas automatizadas. La arquitectura basada en componentes Blazor, servicios y Entity Framework Core mejora la mantenibilidad respecto a una solución donde la lógica estuviera acoplada directamente a la interfaz.

El resultado de testing supera los umbrales indicados en la especificación para la lógica de negocio. La generación de reportes PDF desarrollada dentro del proyecto constituye además una funcionalidad adicional susceptible de la bonificación indicada por la guía.

Como verificación final se ejecutó SonarQube Cloud, obteniendo Quality Gate Passed, calificación A en Security, Reliability y Maintainability, 100% de Security Hotspots Reviewed y 0.0% de duplicación. También se corrigió la advertencia de vulnerabilidad de SQLite. Para la entrega se recomienda mantener el branch master como versión final, completar el archivo .txt con los enlaces de Jira y del repositorio, y conservar evidencias del tablero, historial Git, pruebas, cobertura y análisis de calidad.

Anexo A. Estructura técnica relevante

- Components/Pages: vistas y páginas Blazor.
- Services: lógica de negocio y exportación de reportes.
- Models: entidades y objetos del dominio.
- Data/AppDbContext.cs: contexto de Entity Framework Core.
- Data/quinielas_v2.db: base de datos SQLite.
- SistemaQuinielaMundialistasV2.Tests: proyecto xUnit.
- Docs: documentación incremental y archivo de enlaces de entrega.

Anexo B. Comandos de cobertura

```
dotnet test .\SistemaQuinielaMundialistasV2.Tests\SistemaQuinielaMundialistasV2.Tests.csproj --
collect:"XPlat Code Coverage" --settings .\SistemaQuinielaMundialistasV2.Tests\coverage.runsettings
```

```
$cov = Get-ChildItem .\SistemaQuinielaMundialistasV2.Tests\TestResults -Recurse -Filter
coverage.cobertura.xml | Sort-Object LastWriteTime -Descending | Select-Object -First 1
```